Rapport de Travaux Pratiques

Architectures Logicielles

Module :	Architectures Logicielles
Encadrant :	Pr. BE ELBAGHAZAOUI
Établissement :	ENSA Beni Mellal
Année :	2025 – 2026
Étudiant(e) :	ILYAS MACHHOUL
N° Apogée :	23012747

Chapitre 3

TP3 — Architecture Distribuée

3.1 Objectifs

Objectifs du TP

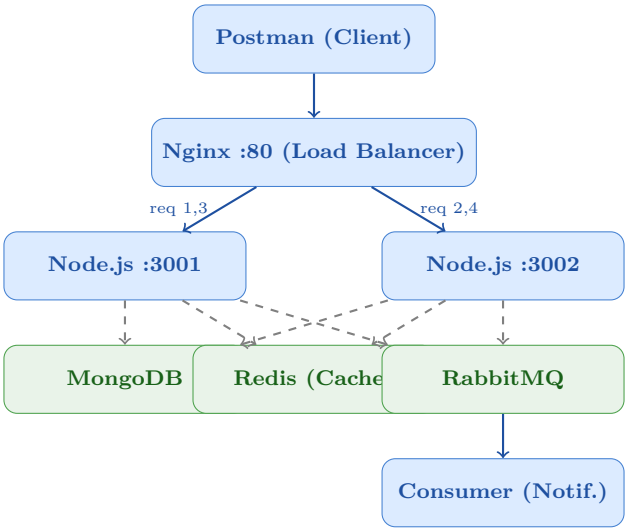
- Déployer une architecture distribuée complète avec **6 composants** interconnectés.
- Comprendre le rôle de chaque composant : Client, Load Balancer, Serveur, Cache, BDD, Bus de messages.
- Implémenter un flux complet : `POST commande` → stockage MongoDB → cache Redis → notification RabbitMQ.
- Orchestrer tous les services avec **Docker Compose**.
- Tester et observer le comportement distribué (Round-robin, cache hit/miss) avec Postman.

3.2 Architecture du Système

Architecture

Composant	Technologie	Rôle
Client	Postman	Envoie les requêtes HTTP POST/-GET
Load Balancer	Nginx	Répartit le trafic entre les instances (Round-robin)
Serveur (×2)	Node.js/Express	Logique métier : validation et traitement
Base de données	MongoDB	Stockage persistant des commandes
Cache	Redis	Mémorise les résultats GET (TTL 30 s)
Bus de messages	RabbitMQ	Notification asynchrone après chaque commande

TABLE 3.1 – Composants de l'architecture distribuée



3.2.1 Structure du projet

```
tp-archi-distribuee/
          docker-compose.yml        # Orchestration de tous les conteneurs
          nginx/
                nginx.conf            # Configuration du Load Balancer
          app/
              Dockerfile
              package.json
              server.js             # Serveur Node.js principal
```

```
9                 consumer.js            # Consommateur RabbitMQ
10                routes/
11                      commandes.js       # Routes POST et GET
12                services/
13                    mongodb.js         # Connexion MongoDB +
              Schema
14                    redis.js           # Connexion Redis
15                    rabbitmq.js        # Connexion RabbitMQ +
              publish
```

Listing 3.1 – Arborescence du projet distribué

3.3 Code Source Important

3.3.1 Docker Compose — Orchestration

```
1  version: '3.8'
2  services:
3    mongodb:
4      image: mongo:6
5      ports:  ['27017:27017']
6      volumes: [mongo_data:/data/db]
7
8    redis:
9      image: redis:7-alpine
10     ports:  ['6379:6379']
11
12   rabbitmq:
13     image: rabbitmq:3-management
14     ports:  ['5672:5672', '15672:15672']
15     environment:
16       RABBITMQ_DEFAULT_USER: admin
17       RABBITMQ_DEFAULT_PASS: admin
18
19   app1:
20     build: ./app
21     environment: { PORT: 3001, INSTANCE: 'Instance-1' }
22     ports: ['3001:3001']
23     depends_on: [mongodb, redis, rabbitmq]
24
25   app2:
26     build: ./app
27     environment: { PORT: 3002, INSTANCE: 'Instance-2' }
28     ports: ['3002:3002']
29     depends_on: [mongodb, redis, rabbitmq]
30
31   nginx:
32     image: nginx:alpine
33     ports: ['80:80']
34     volumes: ['./nginx/nginx.conf:/etc/nginx/nginx.conf']
```

```
    depends_on: [app1, app2]

volumes:
  mongo_data:
```

Listing 3.2 – docker-compose.yml — Extrait principal

3.3.2 Configuration Nginx — Load Balancer

```
events { worker_connections 1024; }

http {
  upstream backend {
    # Round-robin par defaut : req1->app1, req2->app2, req3->app1...
    server app1:3001;
    server app2:3002;
  }

  server {
    listen 80;
    location / {
      proxy_pass          http://backend;
      proxy_set_header    Host        $host;
      proxy_set_header    X-Real-IP   $remote_addr;
    }
  }
}
```

Listing 3.3 – nginx.conf — Round-robin entre les deux instances

3.3.3 Routes API — POST et GET avec cache

```
// POST /commande     Creer une nouvelle commande
router.post('/', async (req, res) => {
  try {
    const { produit, quantite, client } = req.body;

    // 1. Validation
    if (!produit || !quantite || !client)
      return res.status(400).json({ erreur: 'Champs manquants' });

    // 2. Sauvegarde MongoDB
    const commande = new Commande({ produit, quantite, client });
    await commande.save();

    // 3. Invalider le cache (donnees modifiees)
    await redis.del('toutes_commandes');
```

```
16
17      // 4. Publier dans RabbitMQ
18      publierCommande({ id: commande._id, produit, client });
19
20      res.status(201).json({
21        message:       'Commande creee',
22        commande,
23        traitee_par:  INSTANCE
24      });
25    } catch (err) {
26      res.status(500).json({ erreur: err.message });
27    }
28  });
```

Listing 3.4 – commandes.js — Route POST avec MongoDB + Redis + RabbitMQ

```
1  // GET /commandes     Lister avec cache Redis
2  router.get('/', async (req, res) => {
3    try {
4      // 1. Verifier le cache
5      const cached = await redis.get('toutes_commandes');
6      if (cached) {
7        return res.json({
8          source:       'CACHE Redis',
9          traitee_par: INSTANCE,
10         commandes:   JSON.parse(cached)
11       });
12     }
13
14     // 2. Aller chercher en MongoDB
15     const commandes = await Commande.find().sort({ date: -1
          });
16
17     // 3. Mettre en cache (TTL = 30 secondes)
18     await redis.setex('toutes_commandes', 30,
          JSON.stringify(commandes));
19
20     res.json({
21       source:       'MongoDB',
22       traitee_par: INSTANCE,
23       commandes
24     });
25   } catch (err) {
26     res.status(500).json({ erreur: err.message });
27   }
28 });
```

Listing 3.5 – commandes.js — Route GET avec logique cache Redis

3.3.4 Consommateur RabbitMQ

```javascript
async function demarrerConsommateur() {
  const conn    = await
     amqp.connect('amqp://admin:admin@rabbitmq');
  const channel = await conn.createChannel();
  const QUEUE   = 'commandes';

  await channel.assertQueue(QUEUE, { durable: true });
  console.log('Consommateur en attente de messages...');

  channel.consume(QUEUE, (msg) => {
    if (msg) {
      const commande = JSON.parse(msg.content.toString());
      console.log('NOUVELLE COMMANDE RECUE !');
      console.log('  Client  :', commande.client);
      console.log('  Produit :', commande.produit);
      console.log('  ID      :', commande.id);
      console.log('Email de confirmation envoye (simule)');
      channel.ack(msg); // Accuser reception
    }
  });
}

setTimeout(demarrerConsommateur, 5000); // Attendre RabbitMQ
```

Listing 3.6 – consumer.js — Microservice de notification

3.4 Tests avec Postman

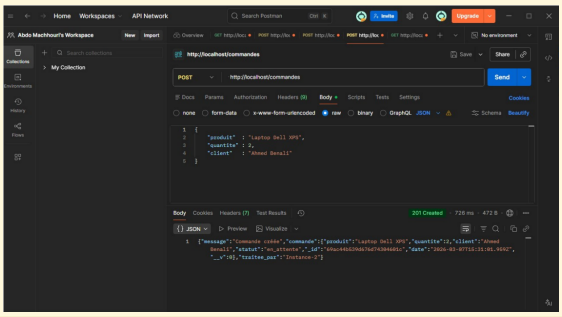
3.4.1 Scénarios de test

Test	Méthode + URL	Observation attendue
Santé Nginx	GET /health	`status: OK, instance: ...`
Créer commande	POST /commandes	`201`, message RabbitMQ affiché
1er GET	GET /commandes	`source: MongoDB`
2ème GET	GET /commandes	`source: CACHE Redis` (instantané)
Load balancing	4 × POST	`traitee_par` alterne I-1/I-2

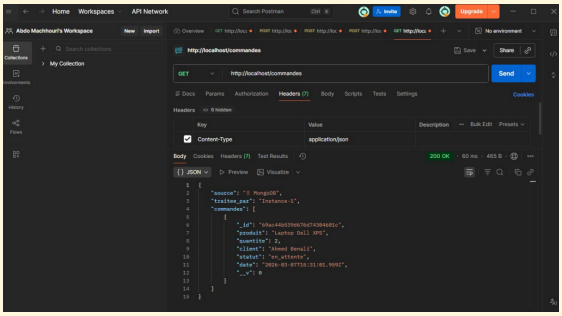
TABLE 3.2 – Scénarios de test — TP3

3.4.2 Captures d'écran Postman

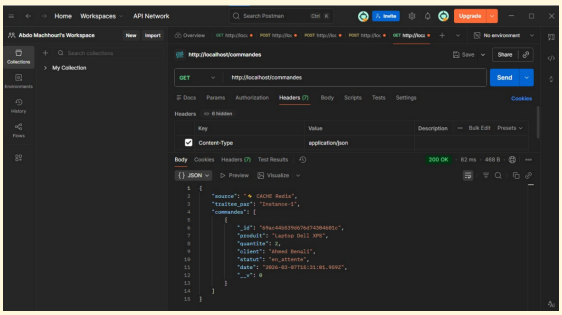
Capture Postman — POST /commandes — Création et réponse 201 (Instance-1)



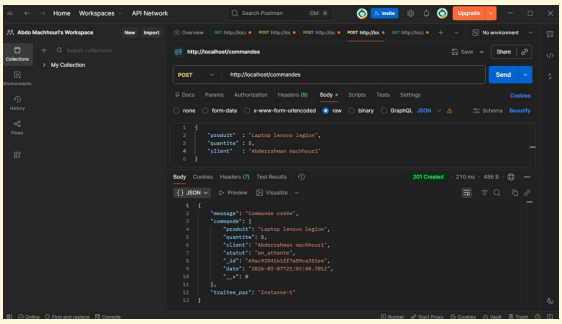
Capture Postman — GET /commandes — 1er appel : source MongoDB



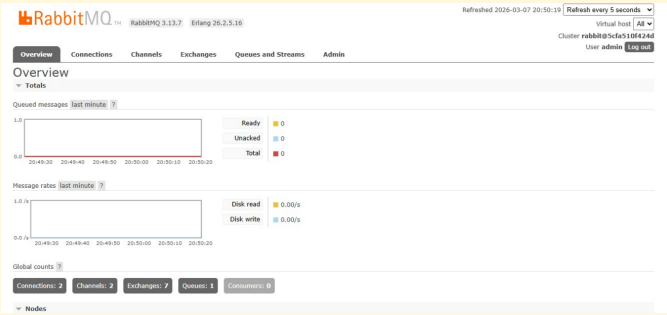
Capture Postman — GET /commandes — 2ème appel : source Cache Redis



Capture Postman — Load Balancing — Alternance Instance-1



Capture Postman — Interface RabbitMQ — http ://localhost :15672



3.5 Questions de Compte-Rendu

1. **Q1 — Quel composant reçoit en premier les requêtes ? Quel algorithme utilise-t-il ?**

   Le premier composant à recevoir les requêtes du client est Nginx, qui joue le rôle de Load Balancer en écoutant sur le port 80. Il utilise l'algorithme Round-Robin qui consiste à distribuer les requêtes en alternance entre les instances disponibles. Ainsi, la première requête est envoyée à Instance-1, la deuxième à Instance-2, la troisième revient à Instance-1, et ainsi de suite de façon cyclique et équitable.

2. **Q2 — Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ?**

   Oui, Nginx continue à fonctionner normalement même si app1 tombe en panne. Lorsque Nginx détecte qu'une instance ne répond plus, il redirige automatiquement toutes les requêtes vers l'instance encore disponible, en l'occurrence Instance-2. Cela est possible parce que Nginx surveille en permanence l'état de santé des instances qui lui sont rattachées. C'est précisément l'un des avantages majeurs

d'un Load Balancer : il garantit la haute disponibilité du système, car tant qu'au moins une instance est active, le service reste accessible aux utilisateurs.

3. **Q3 — Expliquez la différence entre le 1er GET (MongoDB) et le 2ème GET (Redis).**

   Le 1er GET interroge directement la base de données MongoDB, ce qui implique une lecture sur disque et donc un temps de réponse plus élevé. Le 2ème GET est servi depuis le cache Redis, qui stocke les résultats en mémoire vive. Redis étant beaucoup plus rapide que MongoDB, le temps de réponse est

---

4. **Q4 — Pourquoi invalide-t-on le cache Redis après un POST /commande ?**

   Le cache Redis est invalidé après un POST /commands afin de maintenir la cohérence des données. En effet, si le cache n'était pas invalidé, les prochaines lectures retourneraient des données obsolètes ne reflétant pas les nouvelles entrées insérées dans MongoDB. L'invalidation force donc une nouvelle lecture depuis MongoDB lors du prochain GET

---

5. **Q5 — En quoi RabbitMQ améliore-t-il la résilience du système ?**

   RabbitMQ améliore la résilience du système en permettant un traitement asynchrone des événements. Les messages sont publiés dans une file d'attente et persistent même si le consommateur est temporairement indisponible. Lorsque le consommateur redémarre, il reprend le traitement des messages sans perte de données, garantissant ainsi la fiabilité et la non-perte d'événements dans le système

---

6. **Q6 — Pour ajouter une 3ème instance, quelle ligne modifier dans nginx.conf ?**

   Pour ajouter une troisième instance Node.js, il suffit d'ajouter une seule ligne dans le bloc upstream du fichier nginx.conf, à savoir server app3 :3003 ; juste en dessous des deux serveurs existants. Il faudrait également déclarer le nouveau service app3 dans le fichier docker-compose.yml en lui attribuant le port 3003 et la variable d'environnement INSTANCE : 'Instance-3'. Une fois ces modifications appliquées, Nginx répartira les requêtes de façon cyclique entre les trois instances, ce qui augmentera la capacité de traitement du système et renforcera davantage sa tolérance aux pannes.